Rapport de projet

IA et Jeux - Projet 3

Loïs ZHU
Chenye LIN

6 mai 2026

Introduction

Ce projet porte sur le problème d'affectation des étudiants dans le master informatique de Sorbonne Université, comportant 10 parcours (AI2D, BIM, CCA, IMA, MIND, QI, RES, SAR, SESI, STL). L'objectif est d'implémenter et d'analyser différentes variantes de l'algorithme de Gale-Shapley, puis d'étudier des modèles d'optimisation basés sur la programmation linéaire en nombres entiers (PLNE).

Nous comparons :

- les algorithmes de Gale-Shapley (côté étudiant et côté parcours),
- des approches optimisant l'équité et l'utilité globale.

Descriptions des fichiers

- Fichier de tests : *main.py*
- Fichiers contenant les fonctions implémentées pour chaque partie du TME :
  - *tme1.py* :
    - `lire_preferences_etu(f)` : lit le fichier d'entrée et retourne une matrice sur les préférences des étudiants.
    - `lire_preferences_spes(f)` : lit le fichier d'entrée et retourne une matrice sur les préférences des parcours.
    - `gale_shapley_etu(prefEtu, prefSpe, capacites)` : implémente l'algorithme de Gale-Shapley côté étudiants.
    - `gale_shapley_spe(prefEtu, prefSpe, capacites)` : implémente l'algorithme de Gale-Shapley côté parcours.
    - `trouver_paires_instables(prefEtu, prefSpe, capacites, affectations_etu)` : détecte les paires instables dans une affectation donnée.
  - *tme2.py* :
    - `generer_pref_etu(n, m)` : génère aléatoirement les préférences des étudiants.
    - `generer_pref_spe(n, m)` : génère aléatoirement les préférences des parcours.
    - `generer_capacites(n, m)` : génère aléatoirement des capacités pour les parcours.
    - `mesurer_temps()` : mesure le temps d'exécution des algorithmes.
    - `mesurer_iterations()` : compte le nombre d'itérations des algorithmes.
  - *tme3.py* :
    - `resoudre_efficacite(prefEtu, prefSpe, capacites)` : résout le problème en maximisant l'utilité totale.
    - `resoudre_plus_petit_k(prefEtu, prefSpe, capacites)` : détermine la plus petite valeur de k assurant une affectation faisable.
    - `evaluer_affectation(nom_algo, affectation, prefEtu, prefSpe, capacites)` : calcule toutes les métriques d'un algorithme.

Réponses aux questions

1 Problème et affectation

Q2 - Choix des structures de données et complexité pour "côté étudiant"

1. **Trouver un étudiant libre**

   Structure : une pile `etu_libres` qui contient les identifiants des étudiants libres (non affectés). On récupère le prochain étudiant libre avec `.pop()`, et on insère un étudiant libre avec `.append()`.

   Complexité : insertion et suppression en $O(1)$.

2. **Trouver le prochain parcours à qui proposer**

   Structure : un tableau d'entiers `etu_propositions` de taille n (n étudiants) qui stocke, pour chaque étudiant, l'indice du prochain parcours à proposer dans sa liste de préférences `prefEtu[etu_id]`.

   Complexité : accès et mise à jour en $O(1)$.

3. **Trouver la position de l'étudiant i dans le classement du parcours j**

   Structure : une matrice `rang_etu[master_j][etu_i]` où l'on stocke le rang de l'étudiant i à l'indice i pour chaque parcours. Elle est pré-calculée une seule fois au début de l'algorithme.

   Complexité : pré-calcul en $O(n \times m)$, accès en $O(1)$.

4. **Trouver l'étudiant le moins préféré par le parcours j parmi ceux qui lui sont affectés**

   Structure : un tableau de listes `spe_affectations` qui stocke les étudiants admis par chaque parcours.

   Complexité : on parcourt les étudiants admis dans le parcours j (taille $= \mathrm{CAP}_j$, capacité du parcours j), puis on compare leurs rangs via la matrice `rang_etu`. La complexité est en $O(\mathrm{CAP}_j)$ (ou en temps constant dans ce projet car la capacité de chaque master est très petite).

5. **Remplacer un étudiant par un autre dans l'affectation courante d'un parcours j**

   Structure : un tableau de listes `spe_affectations`, comme précédemment.

   Complexité : suppression de l'étudiant le moins bien classé en $O(\mathrm{CAP}_j)$ (ou en temps constant car négligeable) et ajout d'un nouveau en $O(1)$.

Q3 - Algorithme "côté étudiant"

Gale-Shapley "côté étudiant" : ce sont les étudiants qui font les propositions aux masters.

Algorithme

1. Initialisation : au début, chaque étudiant et chaque master est considéré comme libre.

2. Sélection : tant qu'il existe un étudiant libre qui n'a pas encore proposé à tous les masters de sa liste :
   — on choisit un tel étudiant i ;
   — cet étudiant i propose au premier master j de sa liste auquel il n'a pas encore fait de proposition.
3. S'il y a encore de la place dans le master j, l'étudiant i est accepté temporairement dans ce master.
4. Si le master j est déjà plein :
   — ce master regarde dans son affectation courante et trouve l'étudiant i' le moins préféré ;
   — si le master préfère l'étudiant i, il accepte i, et i' devient libre ;
   — sinon, le master rejette la proposition de l'étudiant i.
5. Terminaison : l'algorithme s'arrête quand il n'y a plus d'étudiant libre.

Complexité de l'algorithme "côté étudiant"

— le pré-calcul de la matrice prend $O(n \times m)$;
— la boucle principale effectue au plus $n \times m$ propositions ;
— chaque opération interne est en $O(1)$ ou $O(\text{CAP}_j)$.

$$\text{Complexité totale} = O(n \times m) \Rightarrow O(n^2)$$

Q4 - Algorithme "côté parcours"

Gale-Shapley "côté parcours" : ce sont les masters qui font les propositions, et les étudiants ont le droit de les refuser.

Choix des structures de données et complexité

1. **Trouver un master libre à chaque itération**
   Structure : une pile `spe_libres` qui contient les identifiants des masters qui ont encore de la place.
   Complexité : ajout et suppression en $O(1)$.
2. **Trouver le prochain étudiant à qui faire une proposition**
   Structure : un tableau d'entiers `spe_propositions` de taille m (m masters) qui stocke pour chaque master l'indice du prochain étudiant à proposer dans sa liste de préférences `prefSpe[spe_id]`.
   Complexité : recherche et mise à jour en $O(1)$.
3. **Trouver la position du master j dans le classement de l'étudiant i**
   Structure : une matrice `rang_spe[etu_i][master_j]` où l'on stocke le rang du master j à l'indice j pour chaque étudiant. Elle est pré-calculée une seule fois au début de l'algorithme.
   Complexité : pré-calcul en $O(nb_etu + nb_spe)$, accès en $O(1)$.

4. **Remplacer un master par un autre master j' dans l'affectation courante de l'étudiant**

   Structure : un tableau d'entiers `affectations` qui stocke l'identifiant du master pris par chaque étudiant, -1 si l'étudiant est libre.

   Complexité : accès et mise à jour en $O(1)$.

Algorithme

1. Initialisation : au début, chaque master et chaque étudiant est considéré comme libre.
2. Sélection : tant qu'il existe un master libre (la capacité n'est pas atteinte) qui n'a pas encore proposé à tous les étudiants :
   — on choisit un tel master j ;
   — ce master j propose au premier étudiant i de sa liste auquel il n'a pas encore fait de proposition.
3. Si l'étudiant i n'a pas encore de master, il accepte la proposition.
4. Si l'étudiant i est déjà pris par un autre master j' :
   — si l'étudiant préfère le master j à j', il abandonne l'offre de j' et rejoint j ;
   — sinon, la proposition du master j est rejetée.
5. Terminaison : l'algorithme s'arrête quand il n'y a plus de master libre.

$$\text{Complexité de l'algorithme côté parcours} = O(n^2)$$

Q5 - Résultats obtenus

Affectations côté étudiant :

$$[5, 6, 9, 9, 1, 0, 8, 7, 3, 2, 4, 4, 0]$$

Affectations côté parcours :

$$[5, 6, 9, 9, 1, 0, 8, 7, 3, 2, 4, 4, 0]$$

Q6 - Stabilité de l'affectation

Une paire $(\text{étudiant}_i, \text{master}_j)$ est instable si :

— l'étudiant i préfère le master j à son affectation courante ;

— et le master j a soit une place libre, soit un étudiant affecté qu'il préfère moins que l'étudiant i.

Étapes pour trouver des paires instables

1. Reconstruire l'affectation des masters (une liste d'identifiants des étudiants pour chaque master).
2. Pour chaque étudiant, lister les masters qu'il préfère à son master actuel.
3. Vérifier si ces masters préférés veulent cet étudiant :

— si le master a encore de la place ⇒ paire instable ;
— si le master est plein et préfère cet étudiant à son pire admis ⇒ paire instable.

Note sur la complexité :

Pour des raisons de lisibilité, nous avons utilisé `.index()` pour retrouver les rangs dans la fonction `trouver_paires_instables`. Pour une implémentation optimale sur de très grandes données, il suffirait d'utiliser les matrices de rangs pré-calculées (comme dans nos deux algorithmes précédents) pour effectuer ces comparaisons en $O(1)$.

Conclusion

D'après le test, on observe que les deux algorithmes (étudiant et parcours) retournent la même affectation, et il n'existe pas de paires instables : notre résultat est donc stable.

2 Évolution du temps de calcul

Q8 - Courbe représentant le temps de calcul (moyen) en fonction de n

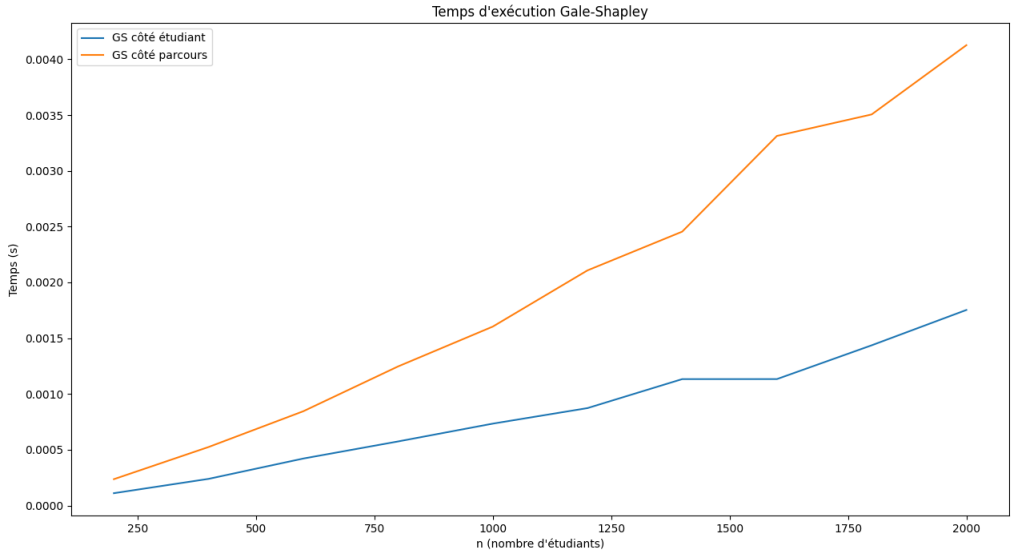

FIGURE 1 – Temps d'exécution de Gale-Shapley en fonction de n

Observation des résultats :

On observe que les deux courbes (Gale-Shapley côté étudiant et côté parcours) augmentent avec le nombre d'étudiants. Les deux algorithmes présentent une croissance similaire, mais l'algorithme côté parcours est plus lent que celui côté étudiant. Cela est dû à la stabilité qui est beaucoup plus longue à atteindre car les rejets sont nombreux en raison du grand nombre d'étudiants qui choisissent entre peu de parcours.

L'écart entre les deux courbes reste globalement le même quand n augmente, ce qui montre que les deux algorithmes évoluent de manière similaire.

Analyse :

Les deux courbes présentent une forme convexe, ce qui correspond à une complexité quadratique. Cela est cohérent avec la complexité théorique de l'algorithme de Gale-Shapley en $O(n^2)$.

Q9 - Complexité de nos algorithmes

L'analyse expérimentale met en évidence une croissance quadratique du temps d'exécution en fonction de n. Cela est conforme à la complexité théorique de l'algorithme de Gale-Shapley, qui effectue au plus $n \times m$ propositions.

Dans notre cas, le nombre de parcours m est constant (égal à 10), ce qui impliquerait une complexité linéaire en $O(n)$ si toutes les opérations internes étaient en temps constant.

Cependant, dans notre implémentation, certaines opérations (comme la recherche du pire étudiant dans un parcours) sont en $O(n)$, ce qui conduit en pratique à une complexité quadratique en $O(n^2)$.

Le côté étudiant et côté parcours présentent donc la même complexité asymptotique, ce qui est confirmé par les courbes obtenues.

Q10 - Analyse du nombre d'itérations

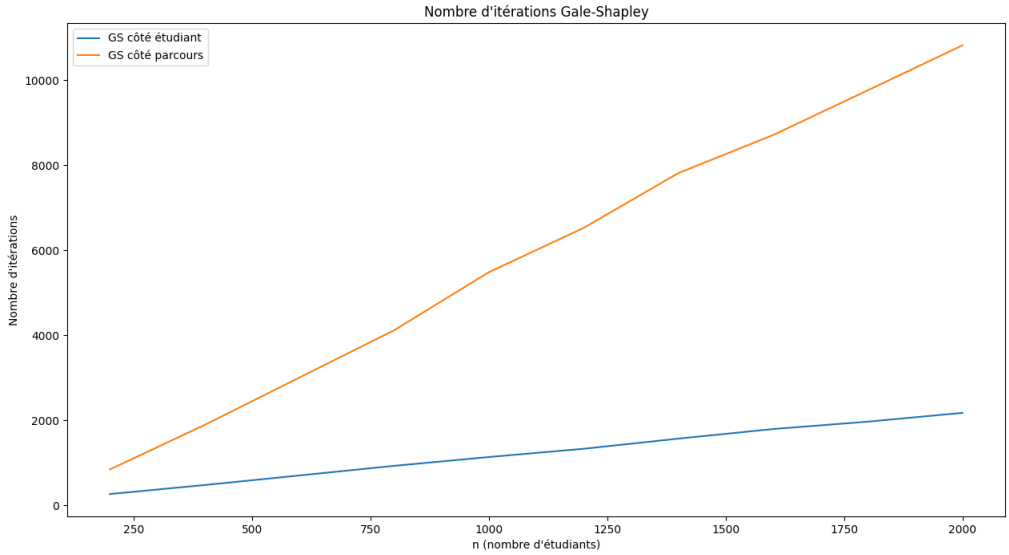

FIGURE 2 – Comparaison du nombre d'itérations selon le côté qui propose

Observation des résultats :

On observe que le nombre d'itérations augmente de manière linéaire avec le nombre d'étudiants et forme une droite pour les deux algorithmes.

Cependant, l'algorithme côté parcours effectue nettement plus d'itérations que celui côté étudiant.

Analyse théorique :

Chaque étudiant peut proposer au maximum à tous les parcours, ce qui donne un nombre total de propositions borné par $n \times m$.

Comme le nombre de parcours est fixé, le nombre d'itérations augmente proportionnellement à n.

Cohérence et justification :

Côté parcours, les masters proposent aux étudiants, mais ces derniers peuvent refuser s'ils ont déjà une meilleure offre. Donc, un même étudiant peut recevoir plusieurs propositions avant d'accepter définitivement, ce qui augmente le nombre total d'itérations.

À l'inverse, côté étudiant, les propositions sont généralement plus efficaces, car les étudiants ciblent directement leurs préférences.

3 Équité et PLNE

On considère :

- n étudiants
- m parcours
- C_j la capacité du parcours j

On définit la variable binaire :

$$x_{ij} = \begin{cases} 1 & \text{si l'étudiant } i \text{ est affecté au parcours } j \\ 0 & \text{sinon} \end{cases}$$

avec :

$$x_{ij} \in \{0, 1\}$$

Les utilités sont définies à l'aide du score de Borda :

$$u_i(j) = m - rang_i(j)$$

où $rang_i(j)$ représente la position du parcours j dans les préférences de l'étudiant i.

Q11 - Maximiser l'équité

On cherche une affectation qui maximise l'utilité minimale des étudiants. Autrement dit, on veut que l'étudiant le "plus malheureux" soit le moins malheureux possible :

$$\max \min_i u_i$$

Comme le minimum n'est pas linéaire, on introduit une variable z représentant l'utilité minimale parmi tous les étudiants (le score du plus malheureux).

Fonction objectif

Maximiser z pour garantir l'équité :

$$\max z$$

Contraintes

Chaque étudiant doit être affecté à exactement un seul parcours :

$$\sum_{j=1}^{m} x_{ij} = 1 \qquad \forall i \in \{1, \ldots, n\}$$

Chaque parcours ne peut pas accueillir plus d'étudiants que sa capacité maximale C_j :

$$\sum_{i=1}^{n} x_{ij} \leq C_j \qquad \forall j \in \{1, \ldots, m\}$$

Définition de l'utilité minimale : l'utilité obtenue par chaque étudiant i doit être supérieure ou égale à z

$$z \leq \sum_{j=1}^{m} u_i(j) x_{ij} \qquad \forall i \in \{1, \ldots, n\}$$

Domaine des variables :

$$x_{ij} \in \{0, 1\} \qquad \forall i, j$$

$$z \geq 0$$

Résultats

- Utilité minimale z : 6.0
- Somme des utilités : 218
- Les scores des étudiants = [9, 10, 10, 10, 10, 10, 10, 10, 6, 10, 6, 9, 10]
- Affectation : [9, 6, 4, 9, 1, 0, 5, 7, 8, 2, 3, 4, 0]

Q12 - Maximiser l'efficacité

On souhaite maintenant maximiser la somme des utilités des étudiants et des parcours. On veut que la somme totale des scores de Borda soit la plus grande possible, peu importe si quelques étudiants se retrouvent avec un mauvais score.

On note :

$$v_j(i)$$

l'utilité du parcours j pour l'étudiant i, calculée par le score de Borda :

$$v_j(i) = n - rang_j(i)$$

où $rang_j(i)$ représente la position de l'étudiant i dans les préférences du parcours j.

Fonction objectif

Maximiser la somme de toutes les utilités (celles des étudiants $u_i(j)$ + celles des parcours $v_j(i)$) pour chaque affectation réalisée :

$$\max \sum_{i=1}^{n} \sum_{j=1}^{m} x_{ij} \left(u_i(j) + v_j(i)\right)$$

Contraintes (identiques à Q11)

Chaque étudiant est affecté à un seul parcours :

$$\sum_{j=1}^{m} x_{ij} = 1 \qquad \forall i \in \{1, \ldots, n\}$$

Respect des capacités des parcours :

$$\sum_{i=1}^{n} x_{ij} \leq C_j \qquad \forall j \in \{1, \ldots, m\}$$

Variables binaires :

$$x_{ij} \in \{0, 1\} \qquad \forall i, j$$

Résolution

On a modélisé à l'aide du solveur Gurobi.

- L'entrée : on donne au solveur Gurobi toutes les utilités (scores de Borda) que chaque étudiant donne à chaque parcours, et inversement. Pour le test, on a pris les matrices qu'on a généréés à la première partie du tme (13 étudiants et 10 parcours).
- Le calcul : pour chaque paire possible (étudiant, parcours), on calcule le score global généré par $u_i(j) + v_j(i)$
- L'objectif : le solveur doit choisir les affectations ($x_{ij} = 1$) de manière à ce que la somme de ces scores soit la plus grande possible.

Analyse des résultats

- Somme des utilités : 239.0
- Utilité moyenne par affectation : 8.77
- Utilité minimale des étudiants : 5
- Les scores des étudiants = [6, 9, 10, 10, 10, 9, 9, 9, 8, 10, 9, 5, 10]
- Affectation : [8, 5, 4, 9, 1, 9, 7, 0, 6, 2, 4, 3, 0]

On remarque que la majorité des étudiants obtiennent un score de 9 ou 10, ce qui montre une excellente satisfaction globale. Par contre, l'utilité minimale est à 5, cela montre que la maximisation de l'efficacité se fait souvent en sacrifiant la satisfaction de certains étudiants pour maximiser le score total.

Q13 - Efficacité sous contrainte d'équité (k premiers choix)

On souhaite maximiser la somme des utilités sous la contrainte que chaque étudiant obtienne au pire son k-ième choix.

Avec le score de Borda, cela revient à imposer que l'utilité de l'étudiant soit au moins égale à $m - k$:

$$u_i(j) \geq m - k$$

Autrement dit, si :

$$u_i(j) < m - k$$

alors l'affectation est interdite :

$$x_{ij} = 0$$

Fonction objectif (identique à Q12)

$$\max \sum_{i=1}^{n} \sum_{j=1}^{m} x_{ij} \left(u_i(j) + v_j(i)\right)$$

Contraintes

Chaque étudiant est affecté à un seul parcours :

$$\sum_{j=1}^{m} x_{ij} = 1 \qquad \forall i \in \{1, \ldots, n\}$$

Respect des capacités des parcours :

$$\sum_{i=1}^{n} x_{ij} \leq C_j \qquad \forall j \in \{1, \ldots, m\}$$

Chaque étudiant reçoit un de ses k premiers choix :

$$\sum_{j=1}^{m} u_i(j) x_{ij} \geq m - k \qquad \forall i \in \{1, \ldots, n\}$$

Variables binaires :

$$x_{ij} \in \{0, 1\} \qquad \forall i, j$$

Q14 - Recherche du plus petit k

On souhaite trouver le plus petit k (k premiers choix) pour lequel PLNE retourne une mariage parfait.

Pour déterminer le plus petit k, on résout successivement le PLNE précédent pour :

$$k = 1, 2, 3, \ldots, m$$

Le plus petit k pour lequel le solveur retourne une solution faisable correspond à la plus petite valeur permettant un mariage parfait.

Résultats

En testant les différentes valeurs de k, le solveur Gurobi a déterminé que pour k=1, 2 et 3, le problème est irréalisable, ce qui signifie qu'il n'existe aucune combinaison permettant de satisfaire tous les étudiants avec leurs 4 premiers choix.

Note sur le décalage de k : étant donné que notre score de Borda pour le premier vœu est égal à m (10 points), il existe un décalage d'une unité. Ainsi, $k = 1$ correspond à l'acceptation du 1er et du 2e vœu ($u \geq 9$). Par extension, $k = 4$ autorise jusqu'au 5ème vœu ($u \geq 6$).

— Le plus petit k trouvé est $k = 4$

— Somme totale des utilités : 227.0

— Les scores des étudiants = [6, 9, 10, 10, 10, 9, 9, 9, 8, 10, 6, 9, 10]

— Affectation : [8, 5, 4, 9, 1, 9, 7, 0, 6, 2, 3, 4, 0]

Q15 - Comparaison des solutions

On compare les solutions obtenues :

— Gale-Shapley côté étudiants

— Gale-Shapley côté parcours

— Solution de la question 11

— Solution de la question 12

— Solution de la question 14

selon les critères suivants :

1. **Stabilité :** on utilise la méthode de la question 6 pour calculer le nombre de paires instables.
2. **Utilité moyenne :**
$$\frac{1}{n}\sum_{i=1}^{n} u_i$$
3. **Utilité minimale :**
$$\min_i u_i$$

Tableau synthétique des résultats

Algorithme / Modèle	Stable ?	Paires Instables	Utilité Min. Étu. (/10)	Utilité Moy. Étu (/10)	Somme Utilité (/260)
GS (Côté Étudiants)	Oui	0	5	8.85	*Non calculé*
GS (Côté Parcours)	Oui	0	5	8.85	*Non calculé*
Q11 (PLNE - Équité)	Non	3	6	9.23	218
Q12 (PLNE - Efficacité)	Non	7	5	8.77	239
Q14 (PLNE - Plus petit k)	Non	5	6	8.85	227

TABLE 1 – Comparaison des algorithmes d'affectation selon les critères de stabilité, d'équité et d'efficacité.

Analyse des modèles

Gale-Shapley et la stabilité

Les algorithmes de Gale-Shapley sont les seuls à garantir une affectation totalement stable (aucune paire instable). Cela signifie qu'aucun étudiant et aucun parcours n'ont intérêt à contourner le résultat.

Dans notre cas, on observe même que les deux versions (côté étudiants et côté parcours) donnent exactement la même affectation. Cela suggère qu'il n'existe probablement qu'une seule solution stable pour ces données.

En revanche, cette stabilité a un coût : l'utilité minimale reste à 5, donc au moins un étudiant n'obtient pas un très bon choix.

Recherche d'équité (Q11)

Le modèle de la Q11 impose d'améliorer la situation du pire étudiant. Ici, l'utilité minimale passe de 5 à 6, ce qui est un vrai gain en équité.

On remarque aussi que l'utilité moyenne des étudiants augmente (elle atteint 9.23), c'est la meilleure parmi tous les tests. Cependant, le score total (étudiants + masters) est le plus bas de tous nos tests (218) et on se retrouve avec 3 paires instables.

On protège donc les individus, mais au prix d'une efficacité globale un peu moins bonne.

Maximisation de l'utilité (Q12)

Le modèle de la Q12 cherche à maximiser la somme des satisfactions. Il obtient effectivement le meilleur score global (239).

Cependant, ce gain se fait au détriment des étudiants : l'utilité moyenne est la plus basse (8.77), et certains restent avec des choix peu satisfaisants. En plus, cette solution n'est pas stable (7 paires instables).

Autrement dit, optimiser le total ne garantit pas une répartition équitable.

Affectation limitée aux k premiers choix (Q14)

Le modèle de la Q14 consiste à limiter les affectations aux k meilleurs choix de chaque étudiant.

Dans notre cas, cela donne un résultat intermédiaire : on garde une utilité minimale élevée (comme en Q11), tout en améliorant la somme totale par rapport à ce modèle.

La solution reste toutefois instable, mais avec un nombre de paires plus limité qu'en Q12.

Conclusion

Au final, le choix dépend de ce que l'on privilégie :

- Si on veut une solution sans contestation : Gale-Shapley et la stabilité
- Si on veut maximiser la satisfaction globale : maximisation de l'utilité
- Si on cherche un équilibre entre équité et efficacité : affectation limitée aux k premiers choix